

OBJECT ORIENTED PROGRAMMING

Design Patterns in

Duolingo



Real-World Software Systems

Arbish Tehseen (CT-25053) · Areefa Samar (CT-25062) · Saita Ahuja (CT-25065)

First Year | Section B · Course Code: CT-260 · Dept. of CS & IT



Composite

Chain of Responsibility

Observer

Flyweight

Memento

Presentation Overview

What we cover today

01

What is Duolingo?

App overview & key features

02

Who Uses It?

User types & demographics

03

Core Modules

Architecture & module mapping

04

Design Patterns

5 GoF patterns explained

05

Code Implementations

C++ for CoR, Composite, Observer

06

UML Diagrams

Structural views of all patterns

What is Duolingo?

Free gamified language learning platform

500M+

Registered Users Worldwide

40+

Languages Supported

8

GoF Patterns in Architecture

Course

Section

Unit

Lesson

Exercise

How It Works

- Short game-like lessons targeting reading, writing, listening & speaking
- Users earn XP points, maintain daily streaks & unlock content
- Weekly leagues with leaderboards for competitive learning
- Duolingo Super removes ads & enables offline learning
- Courses structured as Course → Section → Unit → Lesson → Exercise

Who Are the Users?

A diverse global audience across all ages



Young Learners

Kids & children love the colorful, game-like feel. Easy for any beginner regardless of age.



Students

School & university students practice alongside formal classes to reinforce language skills.



Professionals

Adults learning for travel, job opportunities, or relocation to a new country.



Teachers

Duolingo for Schools lets teachers assign lessons & track student progress.



Global Audience

500M+ users worldwide. Accessible regardless of age, background, or education.

Core Modules & Architecture

8 integrated subsystems — each mapped to an OOP design principle

User Auth & Profile

Encapsulation

Course Catalogue

Composite

Exercise Engine

Factory + Strategy

Gamification

Observer

Notifications

Observer + Strategy

Answer Validation

Chain of Resp.

Session Manager

Memento

Cross-Platform UI

Abstract Factory

T A S K B & C

Design Pattern

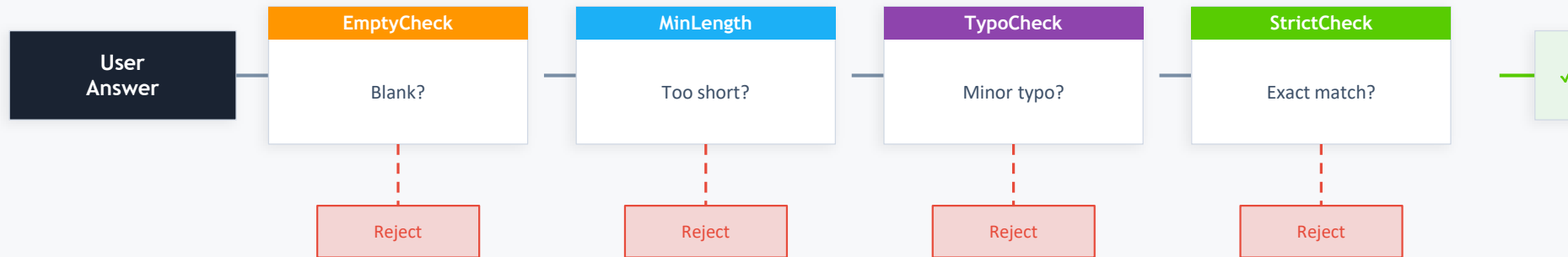
Identification & Implementation



-
- Composite Pattern
 - Chain of Responsibility
 - Observer Pattern
 - Flyweight Pattern
 - Memento Pattern

Pattern 1 — Chain of Responsibility

Behavioral · Answer Validation Pipeline



Abstraction

ExerciseController only knows
AnswerValidator interface

SRP

Each handler has exactly one
validation responsibility

OCP

Add ProfanityFilter as new class
— zero existing changes

Performance

Cheapest checks first —
SemanticNLP runs last only if
needed

Also used in:

User Support Escalation System → AutomatedFAQ → Chatbot → CommunityModerator → HumanSupportAgent



5

Read and respond



If you **promise to do** something, you say that you'll definitely do it.

What does “promise to do” mean?

1

agree to do

2

hate to do

3

know how to do



Good job!

REPORT

CONTINUE



Read and respond



I was so **grateful** when Emma helped me fix my car. I said thank you
ten times!

What does “grateful” mean?

1

wanting to forget someone

2

wanting to thank someone

3

wanting to scream at someone



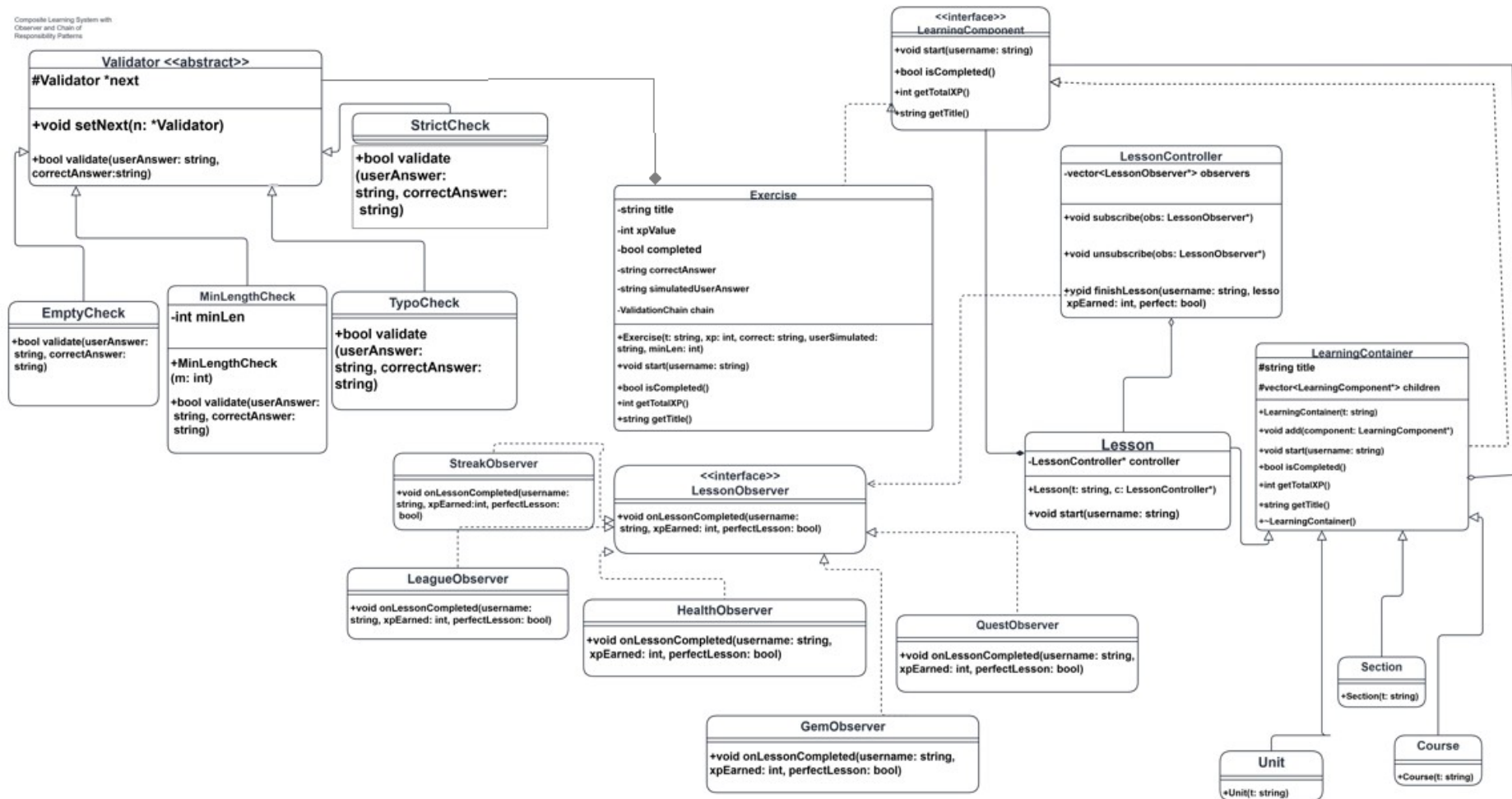
Correct solution:

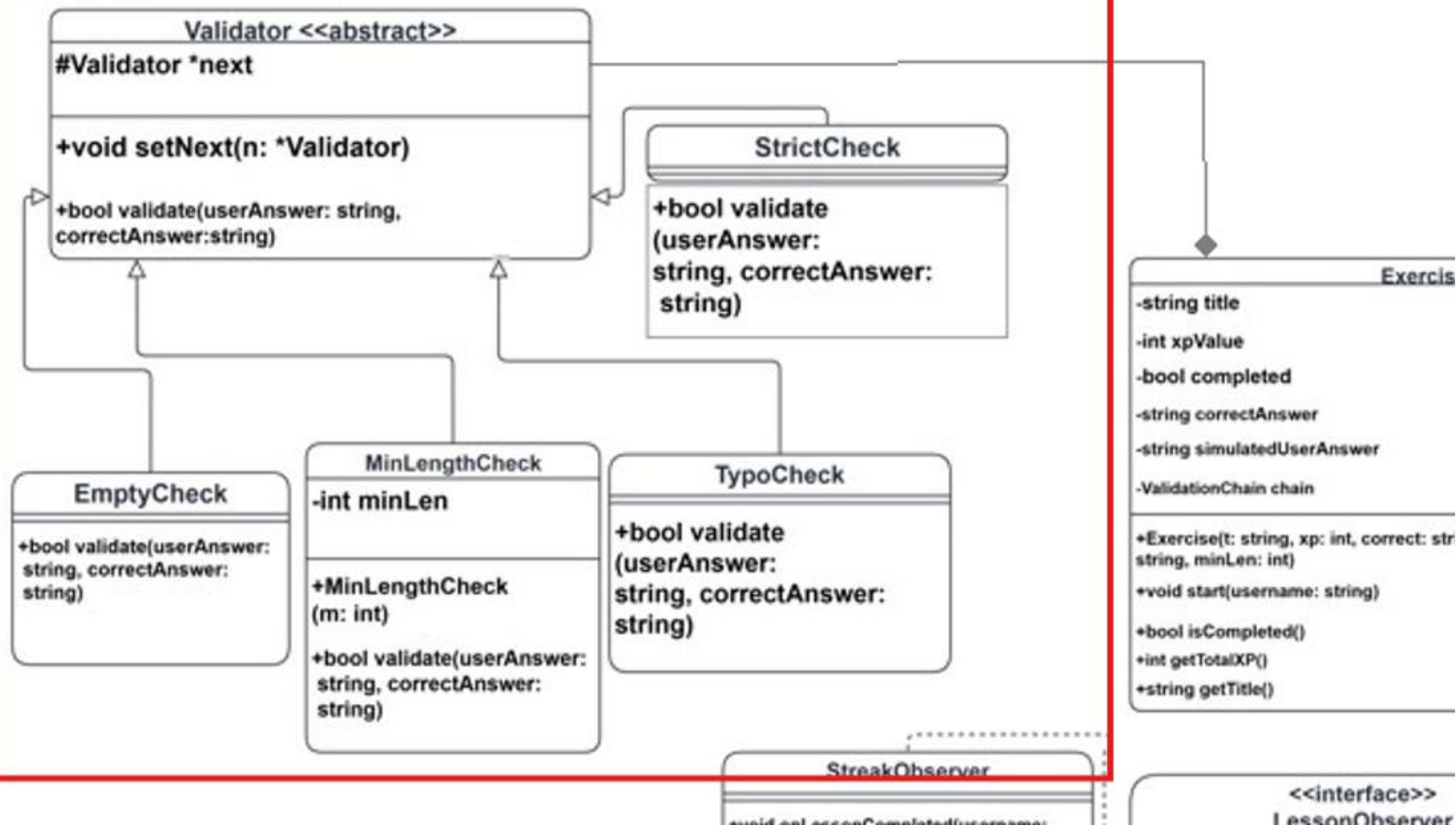
wanting to thank someone



REPORT

CONTINUE





Chain of Responsibility – C++ Implementation

Answer Validation Pipeline · EmptyCheck → MinLength → TypoCheck → StrictCheck

```
// ABSTRACT HANDLER
class Validator {
protected:
    Validator* next;
public:
    Validator() : next(NULL) {}
    void setNext(Validator* n) { next = n; }
    virtual bool validate(string userAns, string correct) = 0;
    virtual ~Validator() {}
};
```

```
// A) Empty Check
class EmptyCheck : public Validator {
public:
    bool validate(string u, string c) override {
        if (u == "") {
            cout << "[EmptyCheck] Rejected: empty.
";
            return false; // stop chain
        }
        if (next) return next->validate(u, c);
        return false;
    }
};
```

```
// B) Min Length Check
class MinLengthCheck : public Validator {
    int minLen;
public:
    MinLengthCheck(int m) : minLen(m) {}
    bool validate(string u, string c) override {
        if ((int)u.size() < minLen) {
            cout << "[MinLength] Rejected: too short.
";
```

```
        return false;
        }
        if (next) return next->validate(u, c);
        return false;
    }
};
```

```
// C) Typo Tolerance (length diff <= 1)
class TypoCheck : public Validator {
public:
    bool validate(string u, string c) override {
        int diff = abs((int)u.size()-(int)c.size());
        if (u != c && diff <= 1) {
            cout << "[TypoCheck] Accepted: minor typo.
";
            return true; // accept here
        }
        if (next) return next->validate(u, c);
        return false;
    }
};
```

```
// D) Strict Final Check
class StrictCheck : public Validator {
public:
    bool validate(string u, string c) override {
        if (u == c) {
            cout << "[StrictCheck] Accepted: exact match!
";
            return true;
        }
        cout << "[StrictCheck] Rejected.
";
        return false;
    }
};
```


Answer Validation Pipeline · EmptyCheck → MinLength → TypoCheck → StrictCheck

```
// MAIN - Build & test the chain
int main() {
    EmptyCheck empty;
    MinLengthCheck minLen(2);
    TypoCheck typo;
    StrictCheck strict;

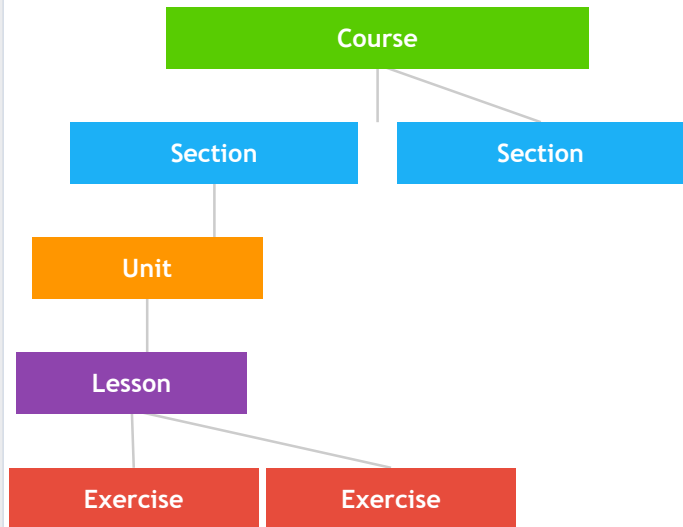
    empty.setNext(&minLen); // empty -> minLen
    minLen.setNext(&typo); // minLen -> typo
    typo.setNext(&strict); // typo -> strict

    empty.validate("", "hello"); // Rejected: empty
    empty.validate("h", "hello"); // Rejected: short
    empty.validate("hell", "hello"); // Accepted: typo
    empty.validate("hello", "hello"); // Accepted: exact
    return 0;
}
```

● Pattern 2 – Composite

Structural · Course Catalogue Hierarchy

Learning Tree Structure



Key Idea

Every node — from a single Exercise to the whole Course — implements the same LearningComponent interface. Clients call start(), getTotalXP(), isCompleted() uniformly on any level.

«interface» LearningComponent

```
+ start()
+ isCompleted()
+ getTotalXP()
+ getTitle()
```

Leaf: Exercise
(no children)

Composite: Lesson, Unit
Section, Course

OCP: Add SkillBlock
→ zero changes elsewhere



LEARN



LETTERS



PRACTICE



LEADERBOARDS



QUESTS



SHOP



PROFILE



MORE

← SECTION 2, UNIT 1

Gratitude: Show appreciation for help



GUIDEBOOK



80



0



385



5

SUPER

Try Super for free

No ads, personalized practice, and unlimited Legendary!



TRY 1 WEEK FREE

LEADERBOARDS

Good job!

You finished #20 and kept your position in the Bronze League



GO TO LEADERBOARDS

Daily Quests

[VIEW ALL](#)



LEARN



LETTERS



PRACTICE



LEADERBOARDS



QUESTS



SHOP



PROFILE



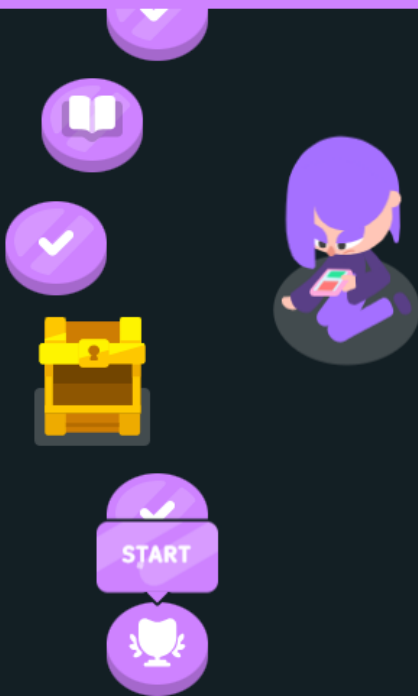
MORE

← SECTION 2, UNIT 2

Gratitude: Respond to unexpected gestures



GUIDEBOOK



80



0



385



5

SUPER

Try Super for free

No ads, personalized practice, and unlimited Legendary!



TRY 1 WEEK FREE

LEADERBOARDS

Good job!

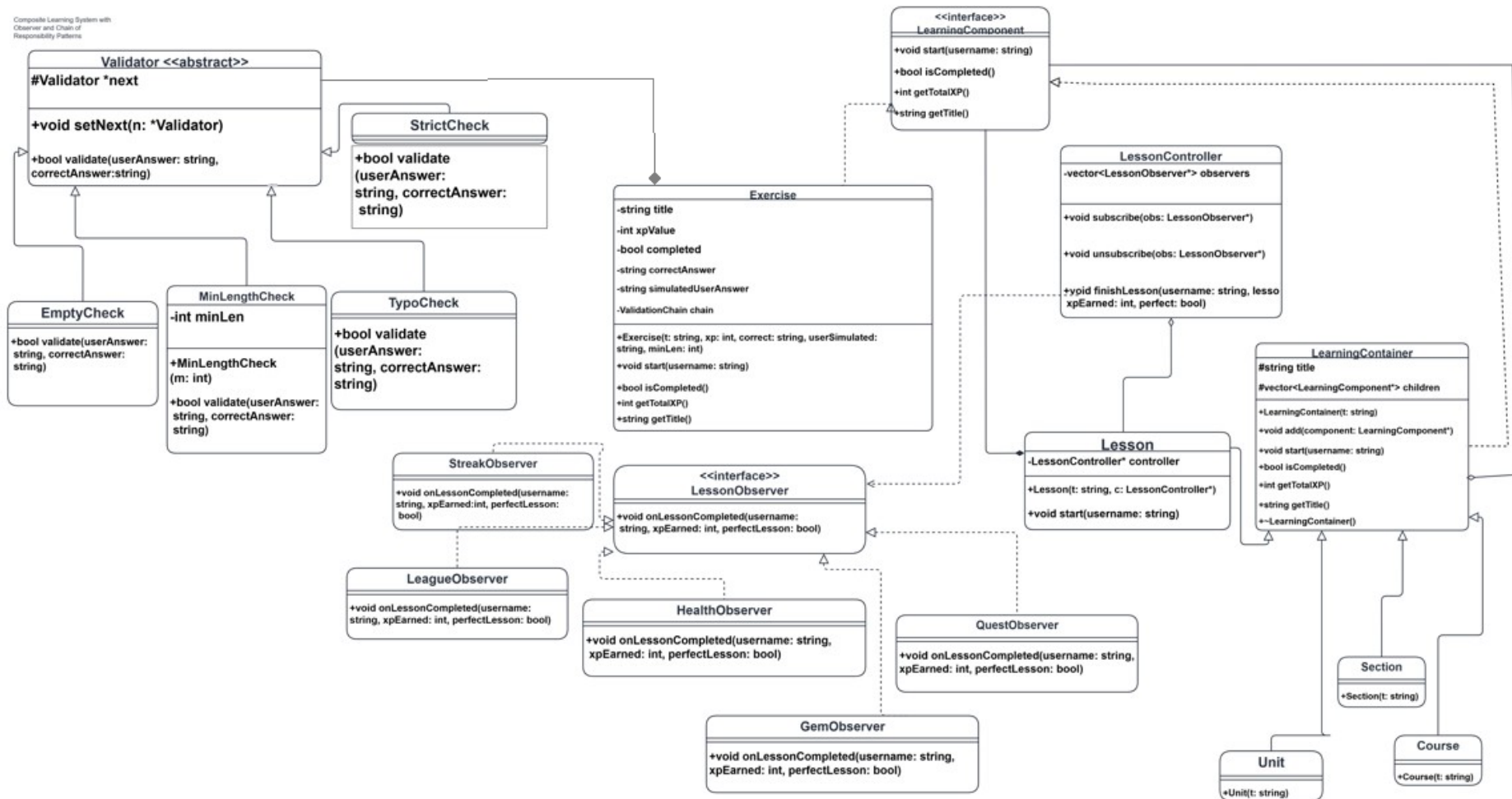
You finished #20 and kept your position in the Bronze League

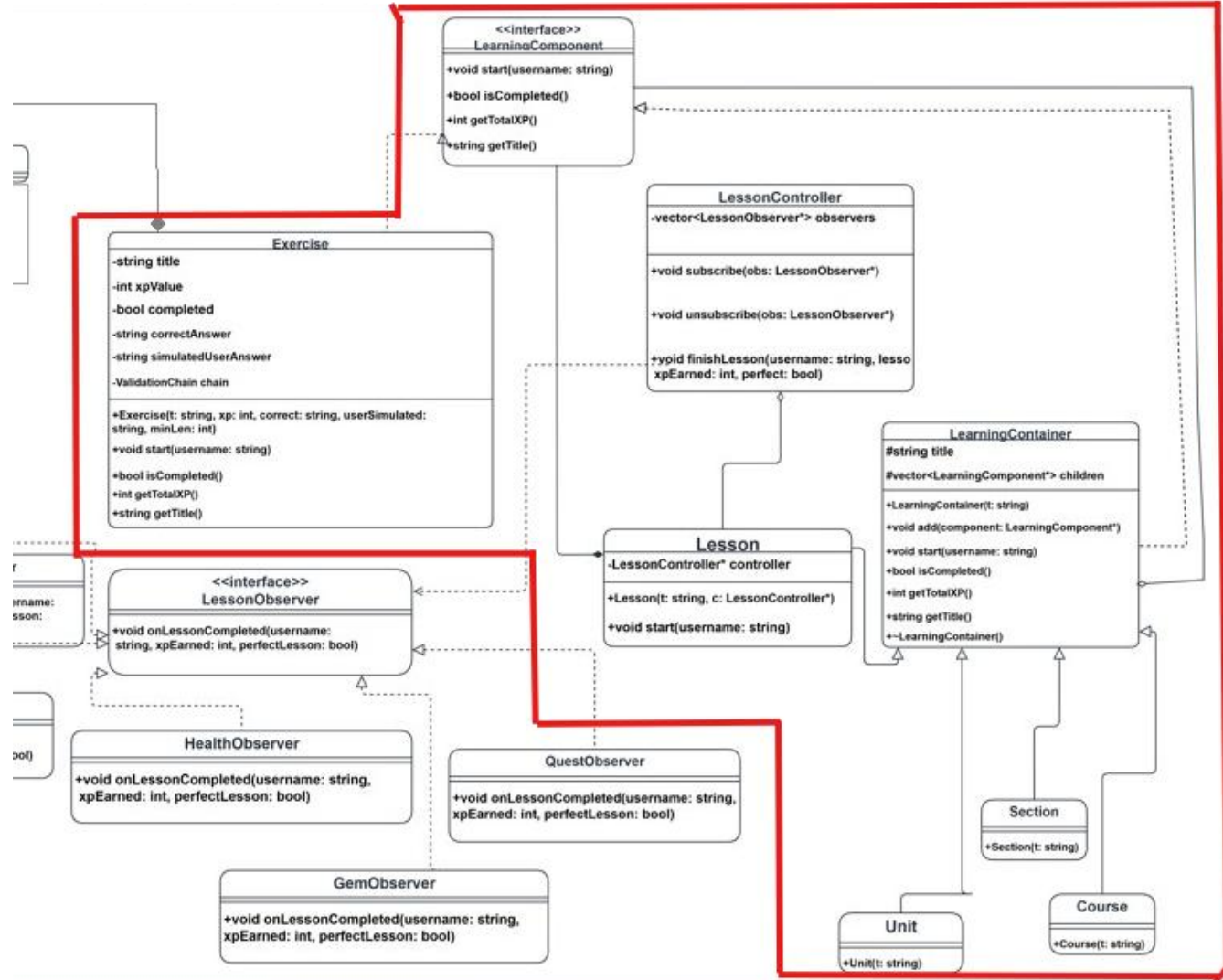


GO TO LEADERBOARDS

Daily Quests

VIEW ALL





Composite — C++ Implementation

Course Catalogue Hierarchy · LearningComponent Interface

```
class LearningComponent {
public:
    virtual void start() = 0;
    virtual bool isCompleted() = 0;
    virtual int getTotalXP() = 0;
    virtual string getTitle() = 0;
    virtual ~LearningComponent() {}
};

class Exercise : public LearningComponent {
private:
    string title;
    int xpValue;
    bool completed;

public:
    Exercise(string t, int xp) : title(t), xpValue(xp), completed(false) {}

    void start() override {
        cout << "        [Exercise] Starting: " << title << "..." << endl;
        completed = true; // Simulating completion
    }

    bool isCompleted() override { return completed; }
    int getTotalXP() override { return xpValue; }
    string getTitle() override { return title; }
};

bool isCompleted() override { return completed; }
int getTotalXP() override { return xpValue; }
string getTitle() override { return title; }
};
```

```
class LearningContainer : public LearningComponent {
protected:
    string title;
    vector<LearningComponent*> children;

public:

    LearningContainer(string t) : title(t) {}

    void add(LearningComponent* component) {
        children.push_back(component);
    }

    string getTitle() override { return title; }

    // Start runs all children recursively

    void start() override {
        cout << "\n>>> Entering " << title << "..." << endl;
        for (auto child : children) {
            child->start();
        }
    }

    bool isCompleted() override {

        for (auto child : children) {
            if (!child->isCompleted()) return false;
        }
        return true;
    }
}
```

Course Catalogue Hierarchy · LearningComponent Interface

```
int getTotalXP() override {
    int total = 0;
    for (auto child : children) {
        total += child->getTotalXP();
    }
    return total;
}

~LearningContainer() {
    for (auto child : children) delete child;
}

};
// --- Specific Composite Types (Hierarchy Levels) ---
class Lesson : public LearningContainer {
public:
    Lesson(string t) : LearningContainer(t) {}
};
class Unit : public LearningContainer {
public:
    Unit(string t) : LearningContainer(t) {}
};
class Section : public LearningContainer {
public:
    Section(string t) : LearningContainer(t) {}
};
class Course : public LearningContainer {
public:
    Course(string t) : LearningContainer(t) {}
};
```

```
int main() {
    Exercise* ex1 = new Exercise("Translate 'Hello'", 10);
    Exercise* ex2 = new Exercise("Match 'Adios'", 5);
    Exercise* ex3 = new Exercise("Speaking: 'Buenos dias'", 15);

    Lesson* lesson1 = new Lesson("Lesson 1: Greetings");
    lesson1->add(ex1);
    lesson1->add(ex2);

    Unit* unit1 = new Unit("Unit 1: The Basics");
    unit1->add(lesson1);

    Section* section1 = new Section("Section: Intro to Spanish");
    section1->add(unit1);

    Course* myCourse = new Course("Spanish for English Speakers");
    myCourse->add(section1);

    // --- EXECUTION ---
    cout << "COURSE TITLE: " << myCourse->getTitle() << endl;

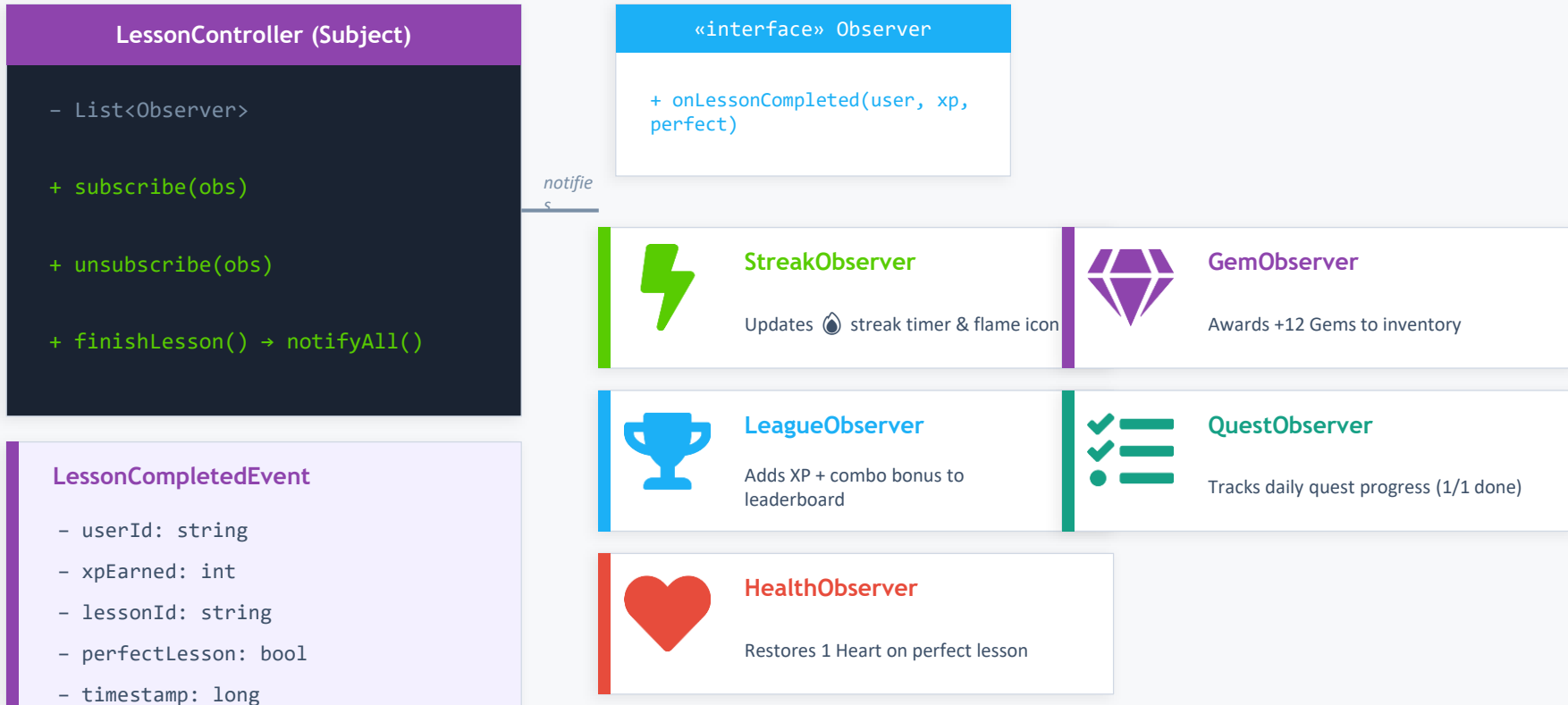
    // Client doesn't care if it's a course or an exercise
    myCourse->start();

    cout << "\n--- FINAL STATS ---" << endl;
    cout << "Total Course XP: " << myCourse->getTotalXP() << " XP" << endl;
    cout << "Course Status: " << (myCourse->isCompleted() ? "Completed! ?" :
    "In Progress... ?") << endl;

    delete myCourse; // Deletes the entire tree
    return 0;
}
```


Pattern 3 – Observer

Behavioral · Gamification & Rewards System





Perfect lesson!

You made no mistakes in this lesson

TOTAL XP

⚡ 40

AMAZING

🎯 100%

REVIEW LESSON

CONTINUE



80

You're a step closer to increasing your English Score!

80



81

CONTINUE



1

day streak

Th

F

Sa

Su

M



But your streak will reset if you don't
practice tomorrow. Watch out!

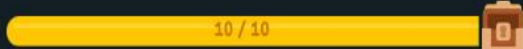
REVIEW LESSON

CONTINUE

All Daily Quests complete!



Earn 10 XP



REVIEW LESSON

CONTINUE



You earned 5 gems!

Nice job reaching your daily goal!

REVIEW LESSON

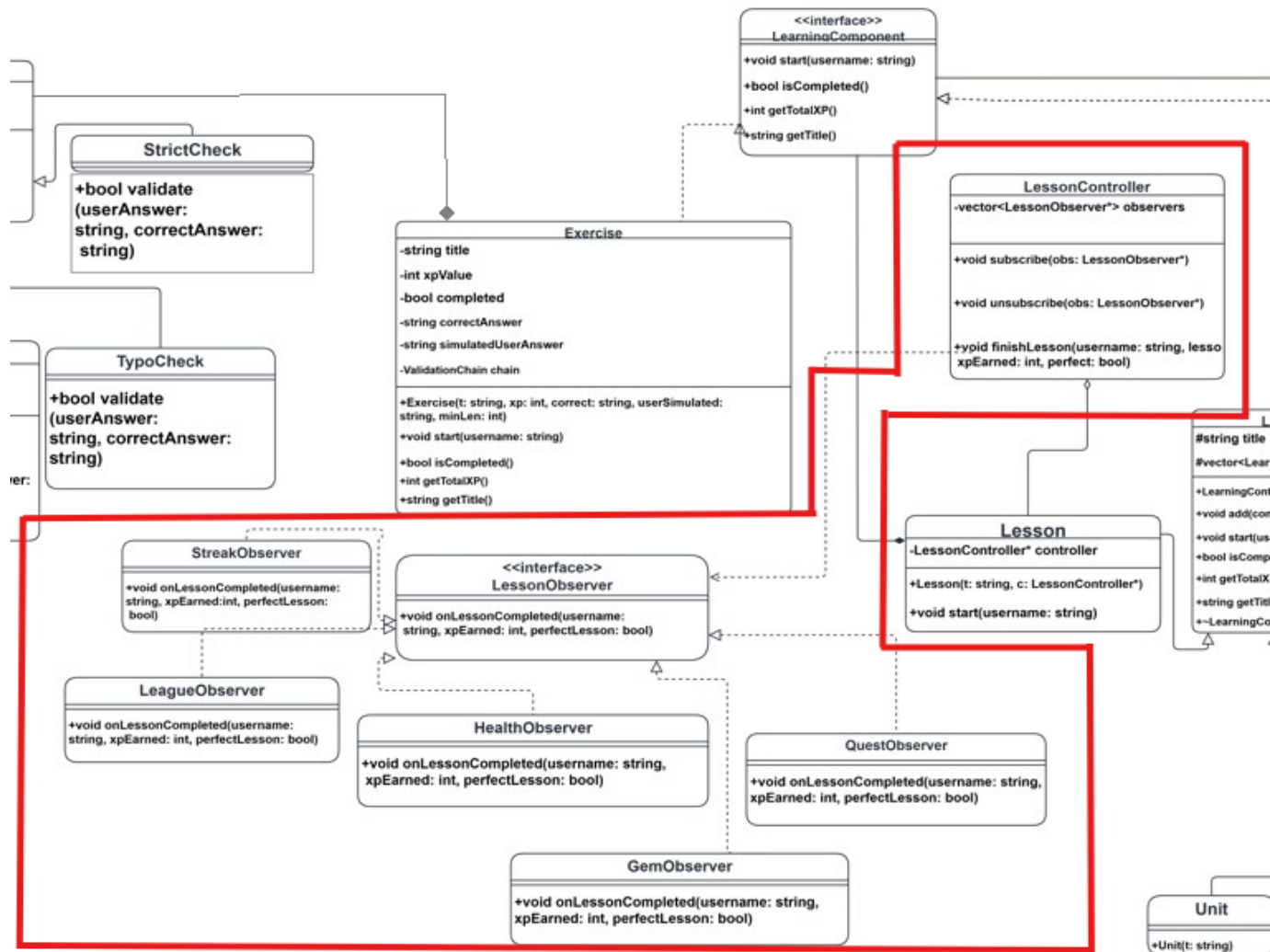
CONTINUE



You completed Unit 2!

REVIEW LESSON

CONTINUE



Observer — C++ Implementation

Gamification System · LessonController notifies all registered observers

```
// OBSERVER INTERFACE
class LessonObserver {
public:
    virtual void onLessonCompleted(
        string username, int xpEarned, bool perfect) = 0;
    virtual ~LessonObserver() {}
};

// Concrete: Streak
class StreakObserver : public LessonObserver {
public:
    void onLessonCompleted(string u, int xp, bool p) override {
        cout << "[STREAK] Flame 🔥 active! Streak = 45 days.
";
    }
};

// Concrete: League (XP + combo bonus)
class LeagueObserver : public LessonObserver {
public:
    void onLessonCompleted(string u, int xp, bool p) override {
        int bonus = p ? 5 : 0;
        cout << "[LEAGUE] +" << xp+bonus << " XP. Rank: 4th
";
    }
};

// Concrete: Health
class HealthObserver : public LessonObserver {
public:
    void onLessonCompleted(string u, int xp, bool p) override {
        if(p) cout << "[HEALTH] Perfect! +1 Heart restored.
";
        else cout << "[HEALTH] Health stable.
";
    }
};
```

```
// SUBJECT: LessonController
class LessonController {
    vector<LessonObserver*> observers;
public:
    void subscribe(LessonObserver* obs) {
        observers.push_back(obs);
    }
    void unsubscribe(LessonObserver* obs) {
        auto it = find(observers.begin(),observers.end(),obs);
        if (it != observers.end()) observers.erase(it);
    }
    void finishLesson(string user, int xp, bool perfect) {
        for (auto obs : observers)
            obs->onLessonCompleted(user, xp, perfect); // notify all
    }
};

// MAIN
int main() {
    LessonController duoEngine;
    StreakObserver streak;
    LeagueObserver league;
    HealthObserver health;

    duoEngine.subscribe(&streak);
    duoEngine.subscribe(&league);
    duoEngine.subscribe(&health);

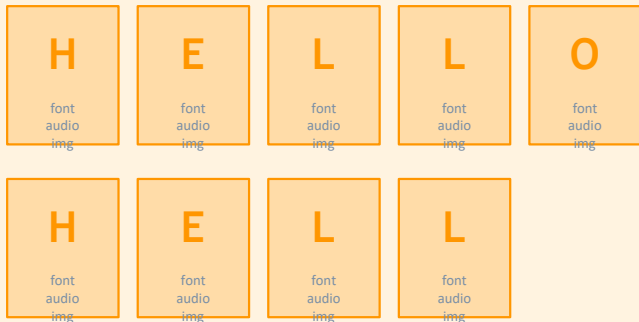
    // Scenario A: Perfect lesson
    duoEngine.finishLesson("Humaira", 15, true);

    // Scenario B: Mute league notifications
    duoEngine.unsubscribe(&league);
    duoEngine.finishLesson("Humaira", 10, false);
    return 0;
}
```

● Pattern 4 — Flyweight

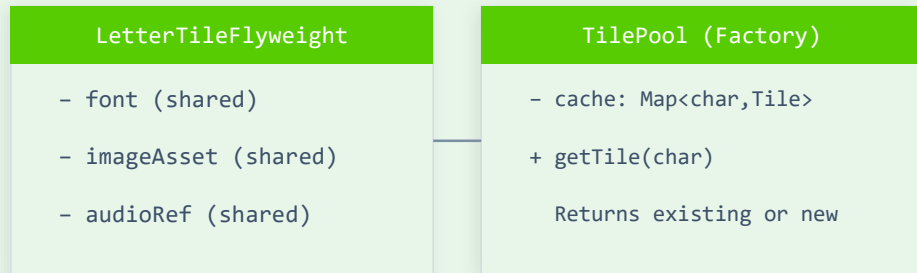
Structural · Letter Tile Renderer

WITHOUT Flyweight ✗



Each tile stores its OWN copy of font, audio, and image asset → massive memory waste on low-end devices!

WITH Flyweight ✓



At Runtime — ExerciseRenderer supplies extrinsic state:



↑ position & selection = extrinsic (passed at runtime, not stored in flyweight)



Type the missing word

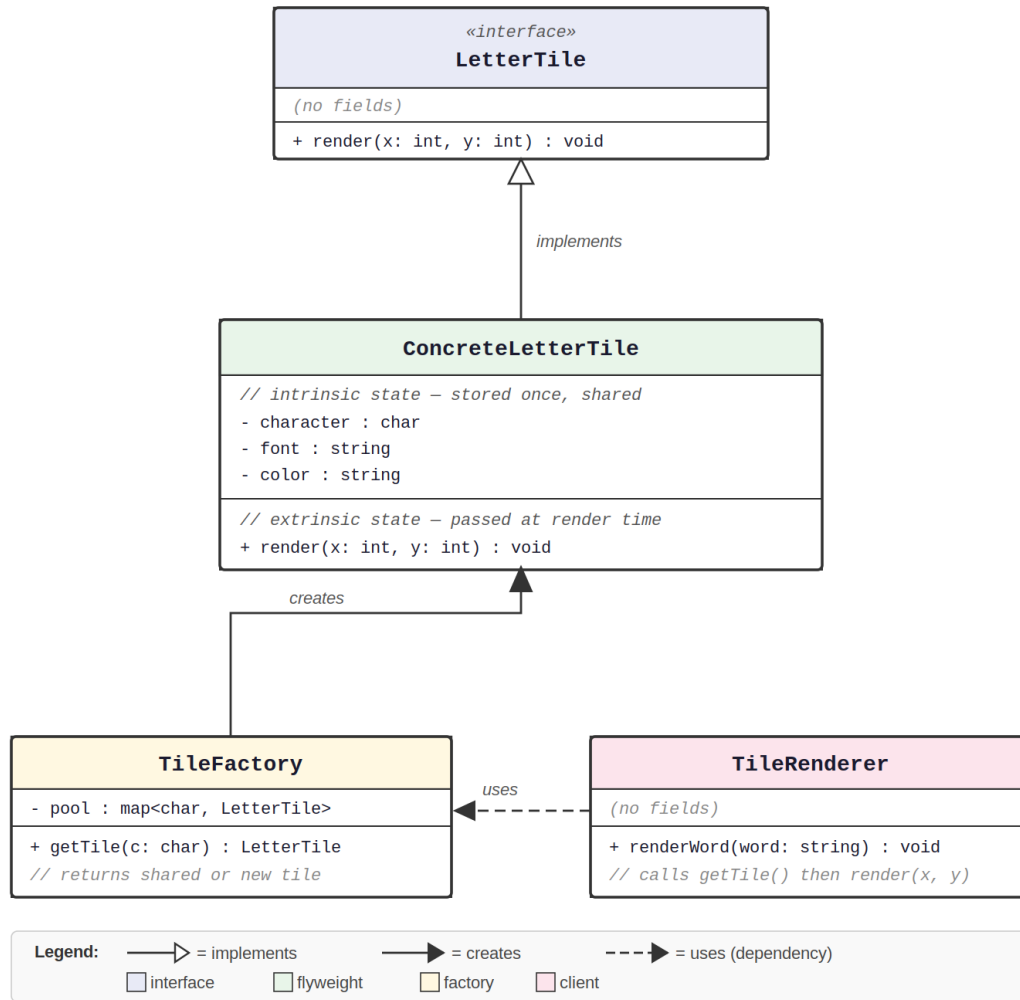


I am de _____ and happy to be here!

CHECK



Flyweight Pattern — Duolingo Letter Tile Renderer

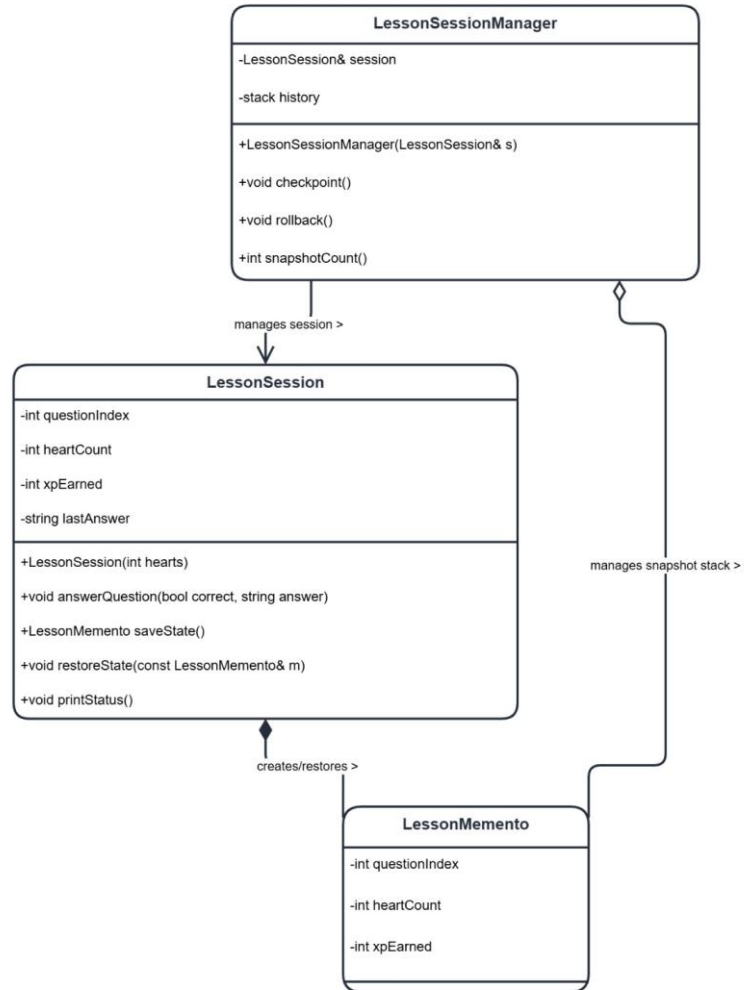


● Pattern 5 – Memento

Behavioral · Session & Progress Manager

LessonSession «Originator»	LessonMemento «Memento (Immutable)»	LessonSessionManager «Caretaker»
- questionIndex - heartCount - xpEarned - answerHistory + saveState() + restoreState()	- questionIndex - heartCount - xpEarned - answerHistory No setters – opaque	- Stack<LessonMemento> + saveCheckpoint() + restoreLastState() Never reads Memento contents

Scenario: User closes app mid-lesson → Caretaker retrieves latest LessonMemento → restoreState() resumes at exact question & streak | Encapsulation preserved.



TASK D

UML Class Diagrams

Structural view of all three implemented patterns

Chain of Responsibility

Validator → EmptyCheck → MinLength → TypoCheck → StrictCheck

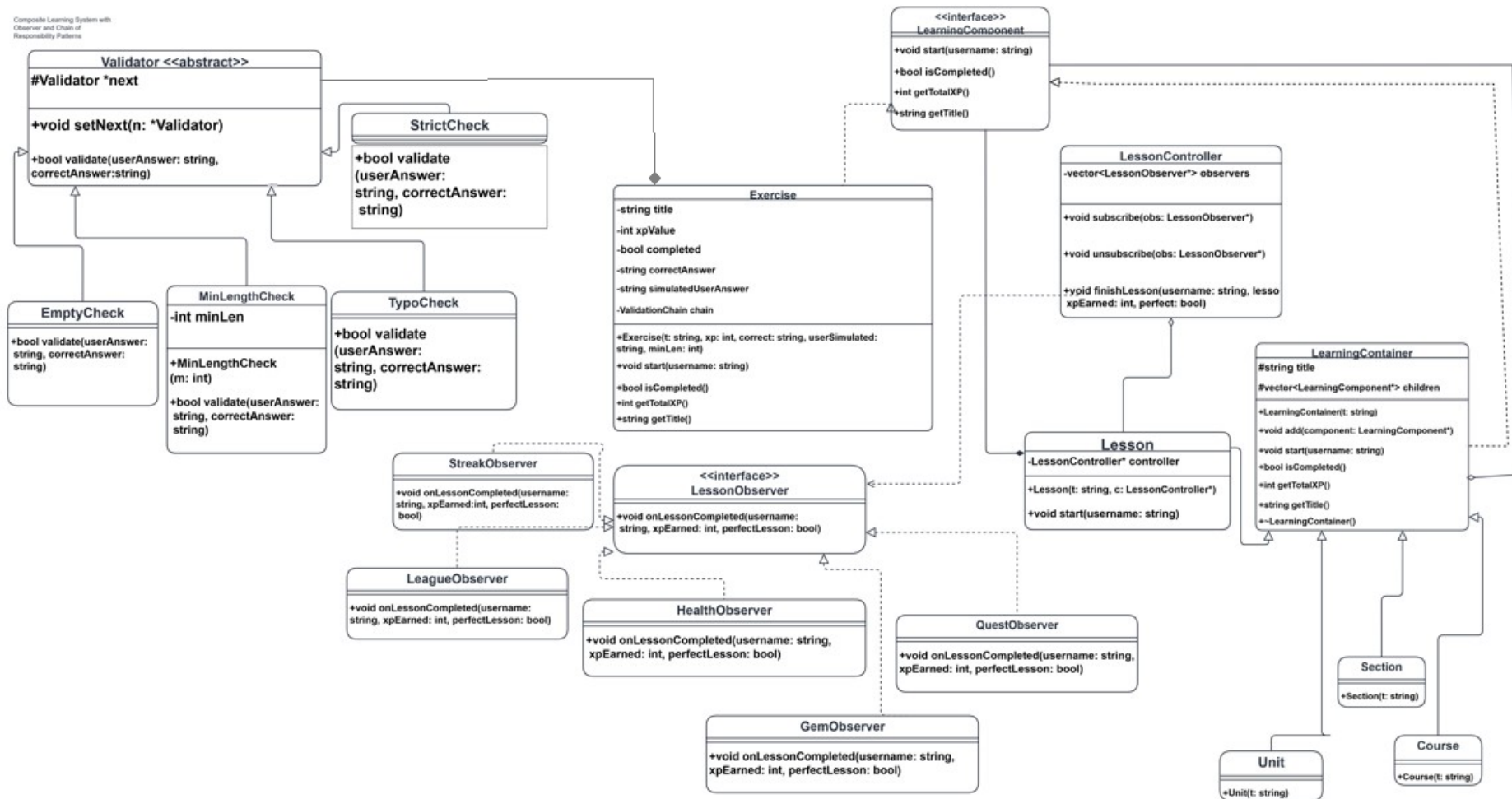
Composite Pattern

LearningComponent ← Leaf(Exercise) & Composites (Lesson, Unit, Section, Course)

Observer Pattern

LessonController ↔ Observer ← StreakObserver, LeagueObserver, HealthObserver...

Note: Combined UML diagrams in report Task D show full class relationships, access modifiers, and inter-pattern connections within the Duolingo architecture.



Design Pattern Summary

All 5 patterns — where they appear in Duolingo

Pattern	Category	Duolingo Module	Key Classes	Status
Composite	Structural	Course Catalogue Hierarchy	LearningComponent, Exercise, Lesson, Unit, Course	C++ Implemented
Chain of Responsibility	Behavioral	Answer Validation Pipeline	Validator, EmptyCheck, MinLength, TypoCheck, Strict	C++ Implemented
Observer	Behavioral	Gamification & Rewards	LessonController, StreakObserver, LeagueObserver...	C++ Implemented
Flyweight	Structural	Letter Tile Renderer	LetterTileFlyweight, TilePool, ExerciseRenderrer	Explained
Memento	Behavioral	Session & Progress Manager	LessonSession, LessonMemento, LessonSessionManager	Explained

Why Duolingo?

6 concrete technical reasons we chose this platform

01

Rich OOP Architecture

Full-stack production system — UI, logic, data & gamification map perfectly onto OOP patterns.

02

8 GoF Patterns — All Visible

Factory, Observer, Flyweight, Composite, Strategy, Abstract Factory, Memento, CoR — one per subsystem.

03

Real-World Scale

500M+ users interact with these exact systems daily. Not academic exercises — live software at scale.

04

Encapsulation Done Right

Sealed Memento, private session data, interface-only access between layers — textbook information hiding.

05

Extensibility as a Feature

Add a new language, exercise, or validator as a new class only. Open/Closed Principle made visible.

06

Relatable Domain

Every student has used a language app. Domain is immediately understood — focus stays on the patterns.

Thank You!

OOP Assignment — Design Patterns in Real-World Software

Arbish Tehseen (CT-25053)

Areefa Samar (CT-25062)

Saita Ahuja (CT-25065)

Dept. of CS & IT · First Year Section B · CT-260



Composite

Chain of Responsibility

Observer

Flyweight

Memento